

Especificación de base de datos y almacenamiento

Escuela Online — esquema actual + ampliaciones requeridas
(video, retención de entregas, mails y chat con IA)

Preparado por: equipo de desarrollo (backend)
Fecha: 25 de agosto de 2026
Repositorio: escuela-app (backend Node.js/Express + knex, frontend React)
Motor actual (desarrollo): SQLite · **Motor recomendado (producción):** PostgreSQL

1. Resumen ejecutivo y alcance

Este documento describe, desde el lado de programación, todo lo que el equipo de DBA/infraestructura necesita para evaluar y aprovisionar las bases de datos y servidores de **Escuela Online**. Cubre dos cosas distintas:

1. El **esquema relacional que ya existe** y corre hoy en desarrollo (14 tablas, sobre SQLite vía knex), documentado tabla por tabla para que sirva de base de partida.
2. **Cuatro ampliaciones nuevas** pedidas explícitamente para esta etapa: almacenamiento de video de los cursos, retención/borrado controlado de las entregas de tareas, registro de emails enviados por la plataforma, y persistencia del chat con IA.

El código de aplicación (controllers, modelos) ya está escrito de forma tal que estas ampliaciones no requieren rediseñar nada existente — se suman tablas y columnas nuevas, y dos piezas del código (`payments.service.js` y `storage.service.js`) ya están armadas específicamente para que conectar servicios externos (pasarela de pago, storage de objetos) sea aislado y no toque el resto de la aplicación. El mismo criterio se extiende acá a video, mail e IA.

Qué se espera del equipo de DBA: evaluar el diseño propuesto (nombres, tipos, índices, relaciones), ajustar lo que corresponda según los estándares internos de la infraestructura, crear las tablas/migraciones en el entorno que corresponda, y aprovisionar los servidores y servicios externos listados en la sección 6 (base relacional, storage de objetos, proveedor de video, SMTP, proveedor de IA).

2. Arquitectura de datos — lineamientos generales

2.1. Motor de base de datos

En desarrollo la app corre sobre **SQLite** (un archivo, cero instalación). Todo el acceso a datos pasa por **knex** (query builder + migraciones versionadas) y ningún controller ni modelo habla con el driver de la base directamente — por eso pasar a producción es cambiar dos variables de entorno (`DB_CLIENT=postgres` , `DATABASE_URL=...`) y correr las migraciones, sin tocar una sola línea de lógica de negocio.

Se recomienda PostgreSQL 15 o superior para producción. Motivo: soporte real de concurrencia de escritura (SQLite serializa escrituras a nivel de archivo, no sirve para múltiples instancias del backend), tipos de datos más ricos (JSON/JSONB si hiciera falta en el futuro), particionado declarativo nativo (relevante para las tablas de crecimiento indefinido de la sección 4.5), y es el motor para el que ya está pensado `knexfile.js`.

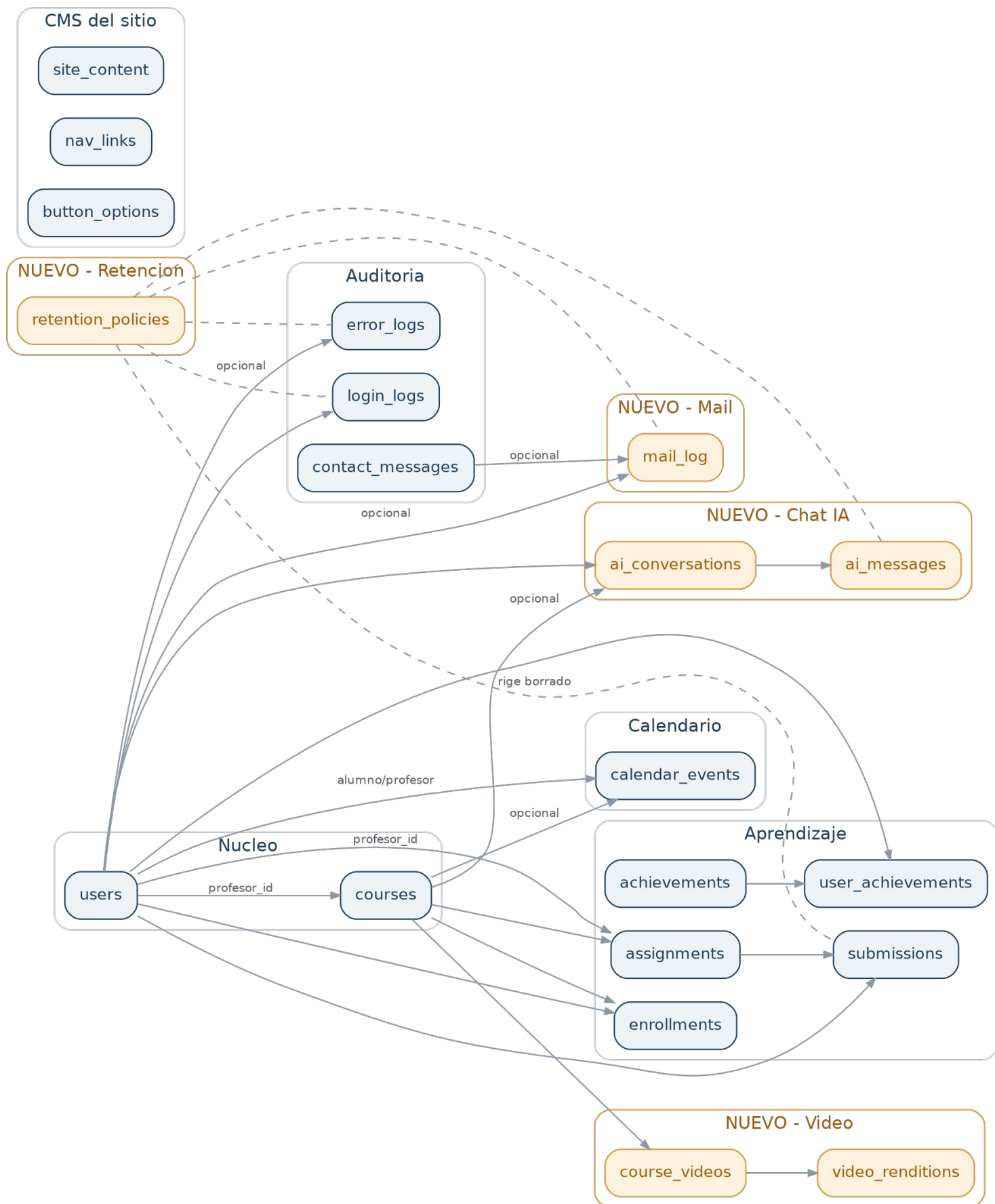
2.2. Relacional vs. almacenamiento de objetos

Regla que ya sigue el código actual y que se mantiene en todo lo nuevo: **la base de datos relacional nunca almacena archivos binarios** (ni video, ni imágenes, ni las entregas de tareas). Guarda únicamente metadata y una referencia (una key de storage o el ID que devuelve un proveedor externo). El archivo en sí vive en almacenamiento de objetos compatible con S3 (o en un proveedor de video administrado, ver sección 4.1). Esto ya está resuelto así para imágenes/entregas de tareas en `backend/src/services/storage.service.js`, que hoy apunta a disco local pero está escrito para poder apuntar a S3 cambiando un solo archivo.

2.3. Convenciones que ya sigue el esquema actual

- Nombres de tabla en `snake_case`, en plural (`users` , `courses` , `submissions` ...).
- Toda tabla tiene `id` autoincremental como PK.
- Los campos de "estado"/"tipo"/"rol" que podrían crecer con el tiempo (agregar un valor nuevo) se guardan como `string` plano, **no** como `ENUM` a nivel de base — la validez de los valores permitidos se controla en la capa de aplicación (`backend/src/utils/roles.js` , `contentTipos.js`). Motivo documentado en el propio código: SQLite no permite alterar un `CHECK` constraint más adelante, así que agregar un estado nuevo dejaría de requerir una migración de "recrear la columna". Se sugiere mantener este mismo criterio en Postgres por consistencia, aunque Postgres sí soporta `ALTER TYPE ... ADD VALUE`.
- Todas las migraciones son archivos versionados con timestamp (`knex migrate:latest`), sin excepción — ninguna tabla se crea "a mano" contra la base.

2.4. Mapa de dominios



Los recuadros naranja son las tablas nuevas de este documento (secciones 4.1 a 4.5). Las flechas punteadas indican qué tablas quedan regidas por una política de retención (sección 4.5), no una relación de datos.

3. Esquema actual (ya en producción de desarrollo)

14 tablas, agrupadas por dominio. Todas corren hoy sobre SQLite vía las 17 migraciones en `backend/src/db/migrations/` y están pobladas por `npm run seed` con datos de prueba.

3.1. Núcleo — usuarios y cursos

users

Columna	Tipo	Null	Detalle
id	serial PK	NO	autoincremental
nombre	varchar	NO	
apellido	varchar	NO	
email	varchar	NO	UNIQUE
password_hash	varchar	NO	bcrypt, 10 rounds
rol	varchar	NO	default 'alumno' — valores válidos: <code>alumno</code> <code>profesor</code> <code>admin</code> (validado en la app, no en la base)
avatar_url	varchar	SÍ	
created_at	timestamp	NO	default now()

courses

Columna	Tipo	Null	Detalle
id	serial PK	NO	
titulo	varchar	NO	
descripcion	text	NO	
precio	decimal(10,2)	NO	default 0
categoria	varchar	SÍ	
imagen_url	varchar	SÍ	ruta en storage, no la base binaria
profesor_id	int	SÍ	FK → users.id, ON DELETE SET NULL
created_at	timestamp	NO	default now()

3.2. Aprendizaje — inscripciones, tareas, logros

enrollments

Columna	Tipo	Null	Detalle
id	serial PK	NO	
user_id	int	NO	FK → users.id, ON DELETE CASCADE
course_id	int	NO	FK → courses.id, ON DELETE CASCADE
payment_status	varchar	NO	default 'pagado' (pago simulado hoy)
payment_method	varchar	NO	default 'simulado'
transaction_id	varchar	SÍ	
purchased_at	timestamp	NO	default now()
completado_at	timestamp	SÍ	lo marca el profesor/admin

UNIQUE(user_id, course_id) — un alumno no puede inscribirse dos veces al mismo curso.

assignments (tareas publicadas por el profesor)

Columna	Tipo	Null	Detalle
id	serial PK	NO	
course_id	int	NO	FK → courses.id, CASCADE
profesor_id	int	NO	FK → users.id, CASCADE
titulo	varchar	NO	
descripcion	text	NO	
fecha_entrega	date	SÍ	

created_at	timestamp	NO	default now()
------------	-----------	----	---------------

submissions (entregas de los alumnos — ver ampliación de retención en 4.2)

Columna	Tipo	Null	Detalle
id	serial PK	NO	
assignment_id	int	NO	FK → assignments.id, CASCADE
user_id	int	NO	FK → users.id, CASCADE
archivo_path	varchar	NO	ruta/key en storage (hoy disco local, ver 2.2)
archivo_nombre_original	varchar	SÍ	
comentario_alumno	text	SÍ	
estado	varchar	NO	default 'entregado' — entregado revisado
calificacion	decimal(4,2)	SÍ	
feedback_profesor	text	SÍ	
entregado_at	timestamp	NO	default now()
revisado_at	timestamp	SÍ	

UNIQUE(assignment_id, user_id) — una entrega por alumno por tarea.

achievements (catálogo fijo de logros) / **user_achievements** (logros obtenidos)

Tabla	Columnas clave
achievements	id, codigo (UNIQUE), titulo, descripcion, icono
user_achievements	id, user_id (FK CASCADE), achievement_id (FK CASCADE), achieved_at — UNIQUE(user_id, achievement_id)

3.3. Calendario

calendar_events (turnos alumno-profesor)

Columna	Tipo	Null	Detalle
id	serial PK	NO	
alumno_id	int	NO	FK → users.id, CASCADE
profesor_id	int	NO	FK → users.id, CASCADE
course_id	int	SÍ	FK → courses.id, SET NULL
motivo	varchar	NO	
starts_at / ends_at	timestamp	NO	
estado	varchar	NO	default 'pendiente' — pendiente aceptada rechazada cancelada
notas_alumno / notas_profesor	text	SÍ	
created_at / updated_at	timestamp	NO	

INDEX(profesor_id, starts_at) — acelera el chequeo de superposición de horarios, que se corre tanto al solicitar como al aceptar un turno.

3.4. CMS del sitio público

Tabla	Columnas clave
site_content	id, clave (UNIQUE), tipo (texto imagen color seleccion boton, app-validado), valor, updated_at
nav_links	id, ubicacion (menu footer), texto, url, orden, timestamps
button_options	id, color_fondo, color_texto, link, timestamps

3.5. Auditoría

Tabla	Columnas clave
error_logs	id, status, mensaje, ruta, metodo, user_id (FK SET NULL), created_at — solo errores 5xx reales
login_logs	id, user_id (FK CASCADE), login_at, logout_at (null = sesión activa) — INDEX(user_id, login_at)
contact_messages	id, nombre, email, telefono, mensaje, created_at

4. Ampliaciones requeridas

Las cuatro ampliaciones pedidas para esta etapa. Cada una se presenta con las tablas nuevas, los cambios a tablas existentes si aplican, y los lineamientos de infraestructura específicos (dónde y cómo se guardan los archivos, quién los sirve, cómo se controla el acceso).

4.1. Video de los cursos NUEVO

Cómo NO guardarlo: nunca como blob en la base relacional, y nunca sirviéndolo directo desde el mismo servidor Node/Express que atiende la API — un archivo de video pesa órdenes de magnitud más que cualquier otro dato de la app, y streaming eficiente (range requests, múltiples calidades según el ancho de banda del que mira) no es algo que el backend actual esté armado para resolver.

Recomendación (menor esfuerzo operativo): usar un proveedor de video administrado — Mux, Cloudflare Stream o Bunny Stream son las opciones más directas. El backend sube el archivo original al proveedor, el proveedor transcodifica a HLS multi-calidad automáticamente y lo sirve por su propio CDN. La base de datos solo guarda el ID que devuelve ese proveedor (`proveedor_asset_id`), nunca el archivo.

Alternativa (más control, más esfuerzo operativo): bucket S3-compatible (AWS S3, Cloudflare R2, Backblaze B2) + un worker que transcodifica con ffmpeg (o un servicio como AWS MediaConvert) generando HLS multi-bitrate + un CDN delante del bucket (CloudFront/Cloudflare). Para este caso sí hace falta la tabla `video_renditions` de abajo, porque hay que trackear cada variante de calidad generada.

course_videos

Columna	Tipo	Null	Detalle
id	serial PK	NO	
course_id	int	NO	FK → courses.id, CASCADE
assignment_id	int	SÍ	FK → assignments.id, SET NULL — si el video es material de una tarea puntual en vez de una clase general
titulo	varchar	NO	
descripcion	text	SÍ	
orden	int	NO	default 0 — orden de reproducción dentro del curso
duracion_segundos	int	SÍ	se completa cuando termina de procesar
estado	varchar	NO	default 'subiendo' — subiendo procesando listo error (app-validado)
proveedor	varchar	NO	ej. 'mux' — mismo criterio que storage.service.js : aislado, para poder migrar de proveedor cambiando un archivo
proveedor_asset_id	varchar	SÍ	el playback ID que devuelve el proveedor — con esto se arma la URL de reproducción, nunca se guarda la URL fija
storage_key	varchar	SÍ	solo si es self-hosted: key del archivo original en el bucket
thumbnail_url	varchar	SÍ	
tamano_bytes	bigint	SÍ	
created_at / updated_at	timestamp	NO	

INDEX(course_id, orden).

video_renditions (solo si se elige la alternativa self-hosted — con un proveedor administrado no hace falta)

Columna	Tipo	Null	Detalle
id	serial PK	NO	
video_id	int	NO	FK → course_videos.id, CASCADE
calidad	varchar	NO	360p 480p 720p 1080p
formato	varchar	NO	default 'hls' — hls mp4
storage_key	varchar	NO	
bitrate_kbps	int	SÍ	

UNIQUE(video_id, calidad, formato).

Control de acceso: el video de un curso pago no debería tener una URL pública fija. El backend arma/firma la URL de reproducción al vuelo en un endpoint tipo `GET /api/courses/:id/videos/:videoId/playback-url` , verificando primero que el usuario esté inscripto (mismo chequeo que ya existe para el classroom). Tanto los proveedores administrados como un CDN delante de S3 soportan URLs firmadas con expiración corta (minutos/horas) para esto.

4.2. Retención y borrado de entregas de tareas NUEVO

Se pide que las entregas (`submissions`) tengan un **tiempo de guardado predeterminado**, configurable, con dos formas de borrado: automático al vencer ese plazo, o manual desde la app. Para esto se agrega una tabla de configuración genérica (que también sirve para

las demás tablas de crecimiento indefinido, ver 4.5) y columnas nuevas en `submissions`.

Columnas nuevas en submissions

Columna	Tipo	Null	Detalle
expira_en	timestamp	SÍ	calculado al entregar: entregado_at + dias_retencion vigente en retention_policies
eliminado_at	timestamp	SÍ	marca de borrado (soft-delete)
eliminado_por	varchar	SÍ	'sistema' 'admin'
eliminado_por_user_id	int	SÍ	FK → users.id, SET NULL — qué admin lo borró manualmente, si aplica

retention_policies (tabla de configuración, no de datos operativos)

Columna	Tipo	Null	Detalle
id	serial PK	NO	
entidad	varchar	NO	UNIQUE — 'submissions' 'error_logs' 'login_logs' 'mail_log' 'ai_messages'
dias_retencion	int	NO	ej. 365 para entregas de tareas
accion	varchar	NO	default 'soft_delete' — soft_delete hard_delete
activo	boolean	NO	default true — permite desactivar la política sin borrar la fila de configuración
updated_at	timestamp	NO	

Por qué una tabla y no una constante en código: así el admin puede ajustar el plazo de retención (por ejemplo, de 12 a 24 meses) desde el panel, sin necesitar un deploy nuevo — mismo criterio ya usado en `site_content` para no hardcodear cosas que van a cambiar.

Cómo se aplica:

1. **Automático:** un job programado (cron — puede ser `node-cron` corriendo dentro del propio backend, o un scheduled task a nivel de infraestructura) recorre `retention_policies` cada entidad activa y, para `submissions`, busca filas donde `expira_en < now()` y `eliminado_at IS NULL`: marca `eliminado_at = now()`, `eliminado_por = 'sistema'`, y borra el archivo real del storage (vía `storage.service.js`). Si `accion = 'hard_delete'`, en vez de marcar soft-delete borra la fila completa.
2. **Manual desde la app:** un admin dispara el borrado de una entrega puntual desde `/admin-panel`; mismo mecanismo, pero `eliminado_por = 'admin'` y `eliminado_por_user_id` con quién lo hizo, sin esperar al job.

Por default se recomienda `soft_delete` (se borra el archivo pero la fila queda, con `eliminado_at` completo) en vez de `hard_delete`, para conservar trazabilidad de auditoría — cuántas entregas existieron, cuándo se borraron y por qué. Un `hard_delete` definitivo de filas muy viejas ya soft-eliminadas puede correrse aparte, como limpieza de espacio, si hace falta.

4.3. Registro de emails enviados NUEVO

Hoy el formulario de contacto guarda el mensaje en `contact_messages` pero no envía ningún email — esta tabla es la base para conectar un envío real (respuestas de contacto, notificaciones de calificación, recordatorios de turno, bienvenida al registrarse, etc.), diseñada como una **cola**: un worker toma las filas `pendiente`, las envía por SMTP o por la API de un proveedor (SendGrid, Resend, Amazon SES), y actualiza el estado.

mail_log

Columna	Tipo	Null	Detalle
id	serial PK	NO	
destinatario	varchar	NO	
remite	varchar	NO	
asunto	varchar	NO	
cuerpo	text	NO	ver nota de retención más abajo
tipo	varchar	NO	app-validado — respuesta_contacto bienvenida recordatorio_turno notificacion_calificacion otro
estado	varchar	NO	default 'pendiente' — pendiente enviado fallido
proveedor	varchar	SÍ	smtp sendgrid ses resend
proveedor_message_id	varchar	SÍ	id devuelto por el proveedor, para trackear entregas/rebotes
intentos	int	NO	default 0
error_detalle	text	SÍ	
contact_message_id	int	SÍ	FK → contact_messages.id, SET NULL — si es respuesta a un mensaje de contacto
user_id	int	SÍ	FK → users.id, SET NULL
created_at	timestamp	NO	default now()

enviado_at	timestamp	SÍ	
------------	-----------	----	--

INDEX(estado) — el worker consulta por estado constantemente. INDEX(created_at) — para el job de retención.

Los cuerpos de email pueden contener datos personales. Esta tabla entra en el mecanismo de retención de 4.2/4.5 — no conviene guardar el HTML renderizado completo indefinidamente. Si el volumen de envíos crece mucho, evaluar guardar solo `template_id` + variables usadas en vez del cuerpo ya renderizado.

4.4. Chat con IA NUEVO

Dos tablas: una conversación agrupa varios mensajes, cada mensaje queda como una fila propia (mismo patrón que cualquier chat: OpenAI, Claude, etc. lo modelan igual).

ai_conversations

Columna	Tipo	Null	Detalle
id	serial PK	NO	
user_id	int	NO	FK → users.id, CASCADE
course_id	int	SÍ	FK → courses.id, SET NULL — si el chat es un "tutor" acotado a un curso puntual
contexto	varchar	NO	default 'general' — general tutor_curso soporte (app-validado)
titulo	varchar	SÍ	generado automáticamente o editable por el usuario
archivada	boolean	NO	default false
created_at / updated_at	timestamp	NO	

INDEX(user_id, updated_at) — para listar "mis conversaciones" ordenadas por actividad reciente.

ai_messages

Columna	Tipo	Null	Detalle
id	serial PK	NO	
conversation_id	int	NO	FK → ai_conversations.id, CASCADE
rol	varchar	NO	user assistant system
contenido	text	NO	
modelo	varchar	SÍ	ej. el identificador del modelo que respondió — útil para auditoría y costos
tokens_entrada / tokens_salida	int	SÍ	para trackear costo por conversación
created_at	timestamp	NO	default now()

INDEX(conversation_id, created_at).

Volumen: esta es, de todas las tablas nuevas, la que más rápido puede crecer (una fila por cada mensaje de cada intercambio). Entra en el mecanismo de retención de 4.5, y es la primera candidata a particionar por fecha si el uso escala (ver 5.4).

Privacidad: el contenido puede incluir datos personales que el usuario escriba libremente. Evaluar cifrado a nivel de columna o del volumen si la política de datos de la escuela lo exige, y no usar estos datos para entrenar modelos de terceros sin consentimiento explícito.

4.5. Mecanismo general de retención

`retention_policies` (definida en 4.2) no es exclusiva de `submissions`: se diseñó para cubrir cualquier tabla de crecimiento indefinido. Se recomienda cargar de entrada una política para cada una de estas:

Entidad	Por qué crece sin límite	Retención sugerida (punto de partida)
submissions	una fila por cada entrega de cada alumno, para siempre	365 días, soft_delete
error_logs	un registro por cada error 5xx real	180 días, hard_delete (son logs técnicos, no hace falta trazabilidad larga)
login_logs	una fila por cada inicio de sesión de cada usuario	365 días, hard_delete
mail_log	una fila por cada email enviado	180 días, soft_delete (o hard_delete del <code>cuerpo</code> solamente, si se separa esa columna)
ai_messages	una fila por cada mensaje de cada conversación	180–365 días según política de privacidad de la escuela, soft_delete

Estos números son un punto de partida razonable, no un requisito fijo — el admin los puede cambiar desde `retention_policies` sin deploy.

5. Requisitos de infraestructura y servidores

5.1. Base de datos relacional

- **Motor:** PostgreSQL 15+.
- **Pooling:** si se despliega con más de una instancia del backend, un pool de conexiones delante (pgbouncer o el que ofrezca el proveedor gerenciado) — el backend abre una conexión por instancia vía knex, y Postgres tiene un límite de conexiones concurrentes bajo por defecto.
- **Tamaño inicial:** para un LMS chico/mediano (cientos a pocos miles de usuarios activos) una instancia gerenciada "small" (del orden de 2 vCPU / 4-8 GB RAM) alcanza para arrancar. Escalar verticalmente primero; considerar una réplica de lectura solo si el dashboard/reporting del admin empieza a pesar sobre la base transaccional.
- **Backups:** automáticos diarios + WAL/point-in-time recovery, retención mínima sugerida 30 días.
- **Migraciones:** siguen siendo `knex migrate:latest`, corridas como paso explícito de deploy — nunca automáticas en cada arranque del server en producción (en desarrollo el backend ya chequea al bootear si hay migraciones pendientes y lo loguea bien visible, pero no las aplica solo — ver `GET /api/health`).

5.2. Almacenamiento de objetos (imágenes, entregas de tareas)

- Bucket S3-compatible (AWS S3, Cloudflare R2, Backblaze B2), separado por entorno (dev/staging/prod).
- Versionado de objetos activado.
- Reglas de ciclo de vida (lifecycle rules) para mover a almacenamiento frío o borrar contenido viejo, coordinadas con las políticas de `retention_policies` — cuando el job de retención borra una fila de `submissions`, también tiene que borrar el objeto correspondiente del bucket.

5.3. Video

- Si se opta por un proveedor administrado (recomendado, ver 4.1): solo hace falta la cuenta/API key del proveedor — no requiere infraestructura propia de transcodificación.
- Si se opta por self-hosted: bucket separado para originales + un worker (cola de jobs, ej. BullMQ/Redis) que dispare ffmpeg o un servicio como AWS MediaConvert, y un CDN delante del bucket de salida.

5.4. Particionado y tablas de crecimiento indefinido

Ninguna de estas necesita particionarse desde el día uno, pero conviene dejarlo planeado: si `ai_messages`, `login_logs`, `error_logs` o `mail_log` superan del orden de unos pocos millones de filas, evaluar particionado declarativo por fecha (mensual) en Postgres — simplifica tanto las consultas por rango de fechas que ya usa el admin (Errores, Logins) como el propio borrado por retención (se dropea la partición vieja entera en vez de un `DELETE` masivo).

5.5. Índices recomendados (además de los ya listados por tabla)

- `submissions(expira_en)` — el job de retención filtra por esta columna.
- `mail_log(estado)` y `mail_log(created_at)`.
- `ai_messages(conversation_id, created_at)`.
- `course_videos(course_id, orden)`.

6. Variables de entorno / secretos a aprovisionar

```
# Base de datos
DB_CLIENT=postgres
DATABASE_URL=postgresql://usuario:password@host:5432/escuela_db

# Storage de objetos (imágenes, entregas de tareas)
STORAGE_DRIVER=s3
AWS_REGION=...
AWS_BUCKET=...
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...

# Video (si se usa un proveedor administrado, ej. Mux)
VIDEO_PROVIDER=mux
MUX_TOKEN_ID=...
MUX_TOKEN_SECRET=...

# Email
SMTP_HOST=...
SMTP_PORT=...
SMTP_USER=...
SMTP_PASS=...
# (alternativa por API en vez de SMTP: SENDGRID_API_KEY / RESEND_API_KEY)

# Chat con IA
AI_PROVIDER_API_KEY=...
```

7. Seguridad y privacidad de datos

- Ningún archivo (video, imagen, entrega) se sirve nunca directo desde disco del servidor de la API en producción — siempre vía storage de objetos + CDN, o el proveedor de video.
- URLs de reproducción de video firmadas, con expiración corta, verificando la inscripción del alumno antes de firmarlas — nunca una URL pública fija guardada en la base.
- Toda tabla de crecimiento indefinido tiene una política de retención configurable (sección 4.5), con soft-delete por default para mantener trazabilidad, y borrado manual disponible desde el admin para poder atender un pedido puntual de un usuario ("quiero que borren mis datos").
- `ai_messages` y `mail_log.cuerpo` son las columnas más sensibles de este documento por volumen de datos personales potenciales — evaluar cifrado a nivel de columna o del volumen si la política de privacidad de la escuela lo requiere.
- Este documento es la base técnica para que infraestructura aprovisione y DBA cree el esquema — no reemplaza una política de privacidad formal ni asesoramiento legal sobre protección de datos.

8. Checklist para el equipo de DBA

- ☐ Aprovisionar instancia de PostgreSQL 15+ (dev/staging/prod) y crear `DATABASE_URL` por entorno.
- ☐ Crear tabla `course_videos` (+ `video_renditions` si se elige la vía self-hosted).
- ☐ Agregar columnas `expira_en`, `eliminado_at`, `eliminado_por`, `eliminado_por_user_id` a `submissions`.
- ☐ Crear tabla `retention_policies` y cargar las 5 políticas iniciales de la sección 4.5.
- ☐ Crear tabla `mail_log`.
- ☐ Crear tablas `ai_conversations` y `ai_messages`.
- ☐ Aprovisionar bucket S3-compatible por entorno, con versionado.
- ☐ Definir proveedor de video (administrado o self-hosted) y aprovisionar credenciales.
- ☐ Definir proveedor de email (SMTP o API) y aprovisionar credenciales.
- ☐ Definir proveedor del chat de IA y aprovisionar la API key.
- ☐ Configurar backups automáticos + retención de 30 días como mínimo sobre la base relacional.
- ☐ Evaluar si conviene particionado por fecha desde el arranque o dejarlo para cuando el volumen lo justifique (sección 5.4).